Tech Hub<FP> euroformac

Centro FP Oficial especializado en **Tech & Business**

Unidad 1

Conceptos generales de la arquitectura aplicaciones web

<IMPLANTACIÓN DE APLICACIONES WEB>

Índice

Objetivos generales de la unidad 3

Introducción 4

1. Aplicaciones web vs. aplicaciones de escritorio 6

2. Arquitectura cliente servidor. Elementos 11

3. Arquitectura de tres niveles 14

3.1. Fundamentos de la Arquitectura de Tres Niveles 14

3.2. Ventajas de la Arquitectura de Tres Niveles 15

4. Protocolos de aplicación más usados: HTTP (Hyper Text Transfer Protocol), HTTPS (Hyper Text Transfer Protocol Secure), FTP 16

5. Resumen 18

6. Glosario 20

7. Bibliografía 23

Objetivos generales de la unidad

- Comprender los principios fundamentales de la arquitectura en las aplicaciones web, incluyendo la estructura básica, el flujo de datos y las consideraciones de diseño.
- Identificar y analizar las diferencias clave entre las aplicaciones web y las aplicaciones de escritorio, incluyendo aspectos como la accesibilidad, la actualización, el almacenamiento de datos y la interacción con el usuario.
- Entender la arquitectura cliente-servidor, incluyendo la función y características de cada elemento (cliente, servidor, comunicación de red), y cómo estos interactúan entre sí.
- Profundizar en la arquitectura de tres niveles, abordando sus componentes (presentación, lógica de negocio, y datos), y cómo estos niveles contribuyen a la escalabilidad y mantenimiento de las aplicaciones web.
- Comprender los protocolos de aplicación más comunes en el desarrollo web, incluyendo HTTP, HTTPS y FTP, sus diferencias, usos, y la importancia de la seguridad y eficiencia en la transferencia de datos.

Introducción

En el dinámico mundo de la tecnología de la información, la arquitectura de las aplicaciones web se ha establecido como un pilar fundamental para el desarrollo de soluciones en línea. Esta arquitectura, que define la estructura y el diseño de las aplicaciones web, es crucial para entender cómo interactúan los componentes de software y hardware en un entorno de Internet.

En primer lugar, es esencial comprender que una aplicación web se compone de dos partes principales: el lado del cliente (front-end) y el lado del servidor (back-end). El front-end es la parte de la aplicación que interactúa directamente con los usuarios, a través de una interfaz gráfica que usualmente se visualiza en un navegador web. Aquí, tecnologías como HTML, CSS y JavaScript juegan un papel clave en la presentación y el comportamiento de la aplicación en el dispositivo del usuario.

Por otro lado, el back-end es donde ocurre la lógica principal de la aplicación. Esta parte se encarga del procesamiento de datos, la gestión de la base de datos, la autenticación de usuarios, y otras funciones cruciales. El back-end se ejecuta en un servidor, y utiliza lenguajes de programación como Python, Java, Ruby, o PHP, junto con sistemas de gestión de bases de datos como MySQL, PostgreSQL, o MongoDB.

Un principio fundamental en el diseño de aplicaciones web es la estructura cliente-servidor. En esta estructura, el cliente (generalmente un navegador web) realiza peticiones al servidor, que las procesa y responde adecuadamente. Esta dinámica es esencial para el funcionamiento de las aplicaciones web y se lleva a cabo mediante protocolos como HTTP o HTTPS. Otra consideración importante en la arquitectura de aplicaciones web es la escalabilidad y el rendimiento. A medida que una aplicación crece

en popularidad, debe ser capaz de manejar un número creciente de solicitudes sin comprometer la velocidad o la experiencia del usuario. Aquí, conceptos como el balanceo de carga, el caching, y la computación en la nube son vitales para asegurar que la aplicación pueda escalar eficientemente.

Finalmente, la seguridad es un aspecto crítico en la arquitectura de aplicaciones web. Proteger los datos del usuario y asegurar la integridad de la aplicación contra ataques cibernéticos es esencial. Esto implica implementar medidas como la encriptación de datos, la autenticación segura, y la protección contra vulnerabilidades comunes como inyección SQL y ataques de cross-site scripting (XSS).

En resumen, la arquitectura de aplicaciones web es un campo complejo y en constante evolución, que requiere un equilibrio entre funcionalidad, rendimiento, escalabilidad y seguridad. Comprender estos conceptos es crucial para el desarrollo y el mantenimiento exitoso de aplicaciones web modernas

1. Aplicaciones web vs. aplicaciones de escritorio



La arquitectura de aplicaciones web es un campo crucial y en constante evolución en el mundo de la tecnología de la información. Esta arquitectura se refiere al marco y los métodos empleados para desarrollar aplicaciones accesibles a través de un navegador web. En este apartado, se exploran los componentes, los patrones de diseño y las mejores prácticas en la arquitectura de aplicaciones web, destacando cómo estos elementos trabajan conjuntamente para crear experiencias de usuario efectivas y eficientes.

- Componentes Clave de la Arquitectura de Aplicaciones Web

  Los componentes claves son los siguientes:

  - Front-End (Cliente): El front-end es la parte de la aplicación web que interactúa directamente con el usuario. Utiliza tecnologías como HTML, CSS y JavaScript para crear interfaces de usuario interactivas y visualmente atractivas.
  - Back-End (Servidor): El back-end se ocupa del procesamiento de la lógica de la aplicación, la gestión de bases de datos y el almacenamiento de datos. Lenguajes como Python, Java, y PHP, junto con sistemas de gestión de bases de datos como MySQL o MongoDB, son comúnmente usados en esta capa.

- Capa de Interconexión: La interconexión entre el front-end y el back-end se gestiona a través de APIs (Interfaces de Programación de Aplicaciones) y protocolos como HTTP/HTTPS.

- Patrones de Diseño en la Arquitectura Web
  - Modelo Vista Controlador (MVC): MVC representa un patrón de arquitectura que divide la aplicación en tres partes interdependientes: el modelo, que alberga los datos; la vista, que corresponde a la interfaz de usuario; y el controlador, que maneja toda la lógica y las operaciones. Esta división ayuda a organizar mejor el código y optimiza tanto el crecimiento como el mantenimiento de la aplicación. (Microsoft, 2023)
  - Programación Orientada a Servicios (SOA): SOA es un enfoque que permite la integración de servicios distribuidos, independientes y reutilizables, lo que facilita la creación de aplicaciones web robustas y escalables.
  - Microservicios: Este modelo arquitectónico implica segmentar una aplicación en una serie de servicios independientes, cada uno operando en su propio proceso. Estos servicios interactúan entre sí a través de protocolos de comunicación eficientes, como el HTTP, garantizando una estructura modular y escalable.

<Definición>



Contrario al enfoque monolítico, donde toda la aplicación se desarrolla como una única unidad indivisible, la arquitectura de microservicios descompone la aplicación en pequeños servicios que se ejecutan de manera independiente. Esto mejora la escalabilidad y la facilidad para actualizar y mantener partes específicas de la aplicación sin afectar el sistema completo

- Mejores Prácticas en Arquitectura Web
  - Seguridad: Implementar medidas de seguridad como la autenticación de usuarios, la encriptación de datos y la protección contra ataques como XSS y CSRF es fundamental.
  - Optimización del Rendimiento: Técnicas como la compresión de datos, la minificación de CSS y JavaScript y el uso de redes de entrega de contenido (CDN) son esenciales para mejorar la velocidad de carga de las aplicaciones web.
  - Responsive Design: Diseñar aplicaciones web que sean estéticamente agradables y funcionales en una variedad de dispositivos y tamaños de pantalla es clave para una amplia accesibilidad.
  - Accesibilidad: Asegurar que la aplicación web sea accesible para personas con diversas discapacidades es no solo una práctica inclusiva sino también un requisito legal en muchas jurisdicciones.

Tanto las aplicaciones web como las de escritorio ofrecen funcionalidades distintas y únicas, adecuadas para diferentes necesidades y preferencias de los usuarios.

En esta sección, se analiza en profundidad las características, ventajas y desventajas de ambas para proporcionar una comprensión clara sobre cuál es la mejor opción según sus necesidades.

Las características son las siguientes:

1. Facilidad de Acceso y Compatibilidad: Las aplicaciones web, accesibles a través de un navegador, ofrecen una gran ventaja en términos de accesibilidad. No requieren instalación y pueden ser utilizadas en cualquier dispositivo con conexión a internet. Esto las

hace ideales para usuarios que valoran la movilidad y la facilidad de acceso desde múltiples dispositivos. (Aguirre, 2023)

Por otro lado, las aplicaciones de escritorio deben ser descargadas e instaladas en un ordenador o laptop específico. Esto limita su acceso a ese dispositivo en particular, pero a menudo ofrece un rendimiento más robusto y una integración más profunda con el sistema operativo y el hardware.

2. Rendimiento y Uso de Recursos: En cuanto al rendimiento, las aplicaciones de escritorio generalmente son más rápidas y eficientes. Están diseñadas específicamente para el hardware en el que se ejecutan, lo que les permite aprovechar al máximo los recursos del sistema, como la CPU y la memoria RAM.

   Las aplicaciones web, aunque son más accesibles, dependen de la velocidad de la conexión a internet y pueden ser menos eficientes en el uso de recursos. Sin embargo, la evolución de tecnologías como HTML5 y JavaScript ha mejorado significativamente su rendimiento.

3. Seguridad y Actualizaciones: La seguridad es un aspecto crucial en cualquier aplicación. Las aplicaciones web se actualizan automáticamente, asegurando que los usuarios siempre tengan la última versión con las correcciones de seguridad más recientes. Además, al estar alojadas en servidores, generalmente cuentan con robustas medidas de seguridad implementadas por el proveedor.

   Las aplicaciones de escritorio, aunque pueden ser seguras, requieren que el usuario se encargue de actualizarlas manualmente, lo que a veces lleva a retrasos en la aplicación de parches de seguridad importantes.

4. Personalización y Funcionalidad: Las aplicaciones de escritorio suelen ofrecer un mayor nivel de personalización y funcionalidad específica. Pueden ser diseñadas a medida para satisfacer las necesidades exactas de un usuario o empresa, aprovechando al máximo las capacidades del dispositivo en el que se ejecutan.

   Las aplicaciones web, si bien son más limitadas en terminales de personalización profunda, ofrecen la ventaja de ser más flexibles y fáciles de actualizar. Esto permite una iteración y mejora continua de la aplicación sin necesidad de intervención del usuario.

5. Coste y Mantenimiento: Desde una perspectiva de coste y mantenimiento, las aplicaciones web suelen ser más económicas a largo plazo. Al estar alojadas en la nube, reducen la necesidad de mantenimiento del hardware y actualizaciones de software por parte del usuario, lo que puede resultar en ahorros significativos.
   En cambio, las aplicaciones de escritorio pueden tener un costo inicial más alto y requieren un mantenimiento continuo del software y el hardware por parte del usuario o del departamento de TI.

2. Arquitectura cliente servidor. Elementos

Esta estructura cliente-servidor, fundamental en el diseño y operación de redes, define la forma en que los datos y recursos se comparten entre distintos dispositivos y usuarios. En este apartado, se estudia los elementos esenciales de la arquitectura cliente-servidor, su funcionamiento, y por qué su entendimiento es crucial para cualquier profesional del sector tecnológico.

- Componentes de la Arquitectura Cliente-Servidor: La arquitectura cliente-servidor se compone de dos partes principales: el cliente y el servidor. El cliente es un dispositivo o programa que solicita servicios o recursos a otro programa, el servidor, que opera en otro dispositivo o el mismo. Por su parte, el servidor constituye el programa o dispositivo encargado de proveer los recursos o servicios requeridos por el cliente.

- Funcionamiento de la Arquitectura Cliente-Servidor: El funcionamiento de esta arquitectura es relativamente sencillo pero poderoso. Un cliente realiza una petición al servidor; el servidor procesa esta solicitud y devuelve la respuesta adecuada. Este intercambio de información se basa en un protocolo de comunicación establecido, que garantiza que ambos, cliente y servidor, entiendan y procesen adecuadamente los datos intercambiados.

- Tipos de Servidores en la Arquitectura Cliente-Servidor: Existen diferentes tipos de servidores según la función que desempeñan. Algunos de los más comunes incluyen:

- Servidores Web: Proporcionan páginas web a los clientes, como navegadores de internet.
- Servidores de Aplicaciones: Ofrecen lógica de negocios o servicios de aplicaciones a las aplicaciones cliente.
- Servidores de Base de Datos: Facilitan a los clientes el acceso y la manipulación de bases de datos.

- Ventajas y desafíos de la Arquitectura Cliente-Servidor: Esta arquitectura ofrece múltiples ventajas, entre las que destacan:
  - Escalabilidad: Permite manejar un creciente número de clientes sin sacrificar el rendimiento.
  - Mantenimiento Centralizado: Las actualizaciones y mantenimientos se realizan en el servidor, simplificando la gestión de las aplicaciones.
  - Seguridad Mejorada: Centralizar los datos en servidores permite implementar medidas de seguridad más robustas.

A pesar de sus ventajas, la implementación de esta arquitectura conlleva ciertos desafíos:

- Manejo de la Carga del Servidor: Un gran número de solicitudes puede sobrecargar el servidor.
- Dependencia de la Red: La arquitectura requiere una conexión de red constante y confiable.
- Seguridad de Datos: La centralización de datos implica un riesgo mayor en caso de un ataque al servidor.

- El Futuro de la Arquitectura Cliente-Servidor: En la era del Internet de las Cosas (IoT) y la computación en la nube, la arquitectura cliente-servidor sigue evolucionando. La integración de tecnologías emergentes como la inteligencia artificial y el aprendizaje automático está ampliando sus capacidades y aplicaciones, lo que promete revolucionar aún más nuestra forma de interactuar con la tecnología.

<Sabías que>



Serverless: Es un modelo de ejecución en la nube donde el proveedor de la nube administra la infraestructura de servidores. Los desarrolladores solo necesitan preocuparse por el código de su aplicación, y el proveedor se encarga de ejecutarlo, escalarlo y mantenerlo. Esto puede reducir significativamente los costos operativos y la complejidad de la gestión de servidores.

3. Arquitectura de tres niveles

La arquitectura de tres niveles es un modelo estructural que se ha consolidado como un estándar en el diseño y desarrollo de sistemas informáticos. Su enfoque modular y escalable permite a las organizaciones implementar soluciones tecnológicas eficientes y robustas, capaces de adaptarse a las necesidades cambiantes del mercado. (IBM, s.f.)

3.1. Fundamentos de la Arquitectura de Tres Niveles



La arquitectura de tres niveles divide un sistema informático en tres capas distintas, cada una con responsabilidades y funciones específicas. Estas capas son:

1. Capa de Presentación: Es la interfaz a través de la cual los usuarios interactúan con el sistema. Su principal objetivo es proporcionar una experiencia de usuario intuitiva y eficiente, presentando los datos y funcionalidades del sistema de manera clara y accesible.
2. Capa de Lógica de Negocio: Constituye el núcleo del sistema, donde se procesan los datos y se ejecutan las reglas y procesos de negocio. Esta capa es crucial para garantizar que las operaciones del sistema se realicen de manera correcta y eficiente.
3. Capa de Datos: Se encarga de gestionar el almacenamiento, recuperación y manipulación de los datos. Esta capa asegura la integridad, seguridad y persistencia de la información utilizada por el sistema.

3.2. Ventajas de la Arquitectura de Tres Niveles

La implementación de una arquitectura de tres niveles ofrece numerosos beneficios, entre los que destacan:

- Escalabilidad y Flexibilidad: Permite modificar o ampliar cada capa de manera independiente, facilitando la adaptación del sistema a las necesidades cambiantes del negocio.
- Mantenimiento y Actualización Simplificados: Al separar las responsabilidades en distintas capas, se simplifica la tarea de mantenimiento y actualización del sistema.
- Mejora del Rendimiento: La distribución de tareas reduce la carga en cada capa, optimizando el rendimiento general del sistema.
- Seguridad Reforzada: Al aislar la capa de datos, se incrementa la seguridad de la información, protegiéndola contra accesos no autorizados.

Para implementar de manera efectiva una arquitectura de tres niveles, es esencial considerar los siguientes aspectos:

- Análisis y Diseño Detallado: Un estudio exhaustivo de los requisitos del sistema y un diseño cuidadoso son fundamentales para garantizar que cada capa cumpla con su función específica de manera eficiente.
- Selección de Tecnologías Adecuadas: La elección de las tecnologías y herramientas apropiadas para cada capa es crucial para el éxito del sistema.
- Integración y Pruebas Rigurosas: Una integración efectiva de las capas y un proceso de pruebas exhaustivo aseguran la fiabilidad y estabilidad del sistema.

4. Protocolos de aplicación más usados: HTTP (Hyper Text Transfer Protocol), HTTPS (Hyper Text Transfer Protocol Secure), FTP

La transferencia de información a través de la red es una parte fundamental de nuestra vida diaria. Los protocolos de aplicación como HTTP (Hyper Text Transfer Protocol), HTTPS (Hyper Text Transfer Protocol Secure) y FTP (File Transfer Protocol) son esenciales para el funcionamiento de Internet.

1. HTTP: La Base de la World Wide Web; El Protocolo de Transferencia de Hipertexto (HTTP) constituye la base de la transmisión de datos en Internet. Este sistema estipula la forma en que los navegadores y los servidores web intercambian mensajes. Lo empleamos a diario, con cada acceso a una página web. Dada su simplicidad y eficiencia, se ha establecido como el protocolo estándar para el intercambio de información en línea.
2. HTTPS: Seguridad y Confianza en la Comunicación; El HTTPS, variante segura de HTTP, añade una capa de seguridad a la transferencia de datos. Utiliza el protocolo SSL/TLS para encriptar la información, protegiendo así los datos sensibles de los usuarios durante su transmisión. Esto es crucial para transacciones en línea, como compras o banca electrónica, donde la confidencialidad y la integridad de la información son primordiales.
3. FTP: Eficiencia en la Transferencia de Archivos; El Protocolo de Transferencia de Archivos (FTP) es fundamental para el movimiento de archivos grandes a través de Internet. A diferencia de HTTP y HTTPS, que están diseñados para la transferencia de texto y archivos multimedia en la web, FTP se especializa en la transferencia eficiente de archivos entre sistemas. Es ampliamente utilizado por

administradores de sistemas y desarrolladores web para la gestión de archivos en servidores.

4. Impacto de HTTP y HTTPS en SEO; En el mundo del SEO (Search Engine Optimization), los protocolos HTTP y HTTPS tienen un impacto significativo. Google favorece los sitios que utilizan HTTPS, considerándolos más seguros para los usuarios. Su implementación no sólo fortalece la seguridad del sitio o aplicación web, sino que asimismo ayuda a mejorar e incrementar su visibilidad y clasificación en los motores de búsqueda.
5. FTP y la Gestión Web; Para los desarrolladores y administradores de sitios web, FTP es una herramienta invaluable. Permite la gestión eficaz de archivos en el servidor, facilitando tareas como la actualización de contenido, la copia de seguridad de datos y la transferencia de archivos grandes. Esta eficiencia y seguridad en la gestión de archivos son esenciales para mantener la integridad y el rendimiento óptimo de un sitio web.

5. Resumen

Este apartado proporciona una visión detallada de varios aspectos cruciales de la tecnología de la información, enfocándose principalmente en la arquitectura de aplicaciones web, comparación entre aplicaciones web y de escritorio, la arquitectura cliente-servidor, la arquitectura de tres niveles, y los protocolos de aplicación más usados.

- Arquitectura de Aplicaciones Web: Se discute la importancia de una arquitectura web bien diseñada para crear sistemas eficientes, seguros y escalables. Se cubren distintos modelos de arquitectura, como Cliente-Servidor, Arquitectura en Capas, y Microservicios, y se mencionan tecnologías clave como HTML/CSS/JavaScript, frameworks de JavaScript, servidores backend, y bases de datos. También se abordan temas cruciales como seguridad, rendimiento, escalabilidad, experiencia del usuario, SEO, y tendencias futuras incluyendo computación en la nube y aplicaciones web progresivas.
- Aplicaciones Web vs. Aplicaciones de Escritorio: Se comparan estas dos formas de aplicaciones, destacando las diferencias en accesibilidad, rendimiento, seguridad, personalización, y costo. Mientras que las aplicaciones web son más accesibles y fáciles de mantener, las aplicaciones de escritorio ofrecen mejor rendimiento y mayor personalización.
- Arquitectura Cliente-Servidor: Se describe esta arquitectura como fundamental en la informática moderna, explicando sus componentes (cliente y servidor), funcionamiento, tipos de servidores, ventajas, desafíos en la implementación, y su evolución en el contexto del Internet de las Cosas y la computación en la nube.

- Arquitectura de Tres Niveles: Se analiza este modelo estructural, dividido en Capa de Presentación, Capa de Lógica de Negocio, y Capa de Datos. Se destacan sus beneficios como escalabilidad, mantenimiento simplificado, mejor rendimiento, y seguridad reforzada, y se subraya la importancia de un análisis detallado, selección adecuada de tecnologías, e integración y pruebas rigurosas para su implementación efectiva.
- Protocolos de Aplicación Más Usados: Se examinan los protocolos HTTP, HTTPS y FTP, explicando su función, importancia y el impacto de HTTP y HTTPS en SEO y de FTP en la gestión web.

6. Glosario

- API (Interfaz de Programación de Aplicaciones): Conjunto de rutinas y protocolos que permite la interacción entre diferente software.

- Aplicaciones de Escritorio: Software que se instala y ejecuta en el ordenador del usuario, trabajando de forma independiente del internet.

- Aplicaciones Web: Programas accesibles a través de un navegador de internet, funcionan en un servidor y no requieren instalación en el dispositivo del usuario.

- Arquitectura Cliente-Servidor: Un modelo de diseño de software donde las tareas se reparten entre proveedores de recursos o servicios (servidores) y solicitantes de estos servicios (clientes).

- Arquitectura de Tres Niveles: Un tipo de arquitectura de software que divide las aplicaciones en tres niveles lógicos: la interfaz de usuario, la lógica de negocio y la base de datos.

- Backend: Parte de la aplicación web que se ejecuta en el servidor, gestionando la base de datos y las operaciones del servidor.

- Base de Datos: Colección organizada de datos, generalmente almacenada y accesible electrónicamente.

- Cliente Web: Un programa, como un navegador, que accede a servicios y recursos de un servidor web.

- Código Fuente: Conjunto de instrucciones y declaraciones escritas en un lenguaje de programación que es entendible y editable por un humano.

- Dominio: Nombre único que identifica a un sitio web en Internet.

- Framework: Conjunto de herramientas y componentes estructurales utilizados para desarrollar y estructurar software.

- Frontend: Parte de la aplicación web que interactúa directamente con el usuario, usualmente consiste en interfaces de usuario y experiencia.

- FTP (File Transfer Protocol): Protocolo utilizado para transferir archivos entre sistemas conectados a una red TCP/IP.

- Hosting: Servicio que permite publicar una aplicación o sitio web en Internet.

- HTTP (Hyper Text Transfer Protocol): Protocolo de comunicación utilizado para transferir información en la web, es la base de cualquier intercambio de datos en internet.

- HTTPS (Hyper Text Transfer Protocol Secure): Versión segura de HTTP, utiliza cifrado para proteger la transferencia de datos.

- Lenguaje de Programación: Lenguaje formal utilizado para escribir instrucciones que pueden ser interpretadas o compiladas en un programa de software.

- Middleware: Software que actúa como puente entre diferentes aplicaciones o entre la aplicación y el hardware.

- Servidor Web: Un software y hardware que utiliza HTTP y otros protocolos para responder a las solicitudes de los clientes web.

- SSL (Secure Socket Layer): Tecnología de seguridad estándar para establecer un enlace cifrado entre un servidor web y un navegador.

7. Bibliografía

Aguirre, S. (2023). *El Corte Inglés*.

IBM. (s.f.). *IBM*.

Microsoft. (03 de 08 de 2023). *Microsoft Learn*.